毕业论文详细框架

基于 FastAPI 与 Vue 的校园二手商品交易平台设计与实现

参考样稿研究 · 三级目录 · 逐节写作任务 · 图表与代码证据映射

项目名称	校园集市
系统类型	Python + Vue 校园二手商品交易平台
前端技术	Vue 3、Vite、Vue Router、Pinia、Axios、Element Plus、ECharts
后端技术	FastAPI、SQLAlchemy、SQLite、PBKDF2、HMAC 令牌
项目规模	C 端 + 管理后台；14 个主要数据实体；49 个 API 接口
生成日期	2026 年 8 月 12 日

用途：论文开题、目录确认、正文写作与系统证据准备

目录

打开 Word 后右键此目录并选择“更新域”，即可生成完整目录。

毕业论文详细写作框架

适用对象：当前工作区内已经完成的“校园集市”项目。系统采用 Python 3.12、FastAPI、SQLAlchemy、SQLite、Vue 3、Vite、Vue Router、Pinia、Axios、Element Plus 与 ECharts，实现 C 端和管理后台，现有 14 个主要数据实体、49 个 API 接口。

使用说明：本框架不是论文正文，而是“逐节写作施工图”。每一节均说明写作目的、核心内容、建议证据、图表安排和容易出现的问题。学校模板、字数、参考文献格式与章节数量如有强制要求，应以学校规范为准。

一、对参考论文的研究结论

1.1 参考论文的整体组织方式

参考文档共 53 页，采用较典型的软件工程类毕业论文结构：封面、诚信声明、中文摘要、关键词、目录、绪论、相关技术、系统分析与设计、系统实现、系统测试、结论、参考文献、致谢与论文评定表。正文按照“提出问题—介绍技术—分析需求—完成设计—展示实现—开展测试—总结展望”的顺序展开，符合信息系统设计与实现类论文的基本叙事逻辑。

参考论文将系统论证材料分成四类：一是研究背景和文献现状；二是用例图、架构图、时序图和 E-R 图等设计证据；三是数据库表、类方法和系统界面等实现证据；四是功能、响应时间、压力与账号安全等测试证据。这个组织方式值得保留。

1.2 参考论文值得借鉴的做法

- 论文结构覆盖软件工程全过程，没有只写页面截图，也没有只讲技术概念。
- 技术章节能够说明技术在系统中的具体用途，而不是只给出百科式定义。
- 需求与设计章节使用用例图、架构图、时序图、E-R 图和数据表形成证据链。
- 实现章节按业务模块展开，每个模块同时描述处理流程和页面结果。
- 测试章节区分功能测试、性能测试和安全性测试，具备结果验证意识。
- 结论部分区分研究成果和未来展望，能够回扣绪论中的问题与目标。

1.3 参考论文需要避免的问题

- 章节编号存在不统一现象，部分一级标题缺少章号或小节号，目录层级与正文层级不完全一致。
- 个别图号重复，例如同一章出现重复的“图 4-4”，图题格式也不完全统一。
- 有些技术性能数字或高并发结论缺少测试环境、样本量和原始结果支撑，容易形成“先有结论、后补证据”。
- 部分内容出现对象名称前后不一致、疑似笔误以及数据库字段解释含混的问题。
- 相关技术内容篇幅较长，但部分内容与后续核心实现联系较弱。
- 参考文献的真实性、时效性和引用位置必须逐条核验，不能只在文末堆放文献。
- 界面截图较多但清晰度不一，部分页面缺少对用户目标、关键操作和实现机制的解释。

1.4 新论文的改写原则

新论文保留参考论文的完整软件工程链路，但统一采用“章—节—小节”三级编号；每一个功能必须能够追溯到需求、设计、接口、数据实体、实现页面和测试用例；技术章节只介绍项目真实使用的技术；性能和安全结论必须来自实际测试；对尚未实现的真实支付、强实名认证、WebSocket 实时消息等能力，明确写成系统边界或未来工作，不得包装成已实现成果。

二、论文定位与题目建议

2.1 推荐题目

优先级	论文题目	适用说明
推荐	基于 FastAPI 与 Vue 的校园二手商品交易平台设计与实现	技术准确，能直接对应当前代码与系统成果
备选	基于 Python 与 Vue 的校园二手商品交易平台设计与实现	表述更宽泛，适合学校要求题目不出现具体框架时使用
备选	面向校园场景的二手商品交易平台设计与实现	更突出业务研究，适合弱化技术栈时使用

建议使用第一项。标题中的“FastAPI”负责体现后端技术特征，“Vue”负责体现前端技术特征，“校园二手商品交易平台”限定研究对象，“设计与实现”明确论文类型。

2.2 研究对象

研究对象不是一般电商平台，而是面向高校学生的单件闲置商品交易平台。系统围绕“同校发现、商品发布、校内沟通、创建订单、面交履约、信用评价、内容审核和风险处理”构建业务闭环，并分别提供学生用户使用的 C 端和运营人员使用的管理后台。

2.3 核心问题

论文应围绕以下问题形成主线：传统校园闲置交易依赖社交群和线下口头沟通，存在信息分散、检索困难、身份可信度不足、交易过程缺少留痕、违规内容难以治理、运营数据不可统计等问题；如何使用前后端分离技术构建统一平台，并通过权限控制、审核机制、订单状态管理、消息与通知、举报处理和数据可视化提升校园交易效率与可管理性。

2.4 研究目标

第一，完成覆盖商品发布、浏览、收藏、沟通、订单、评价与举报的 C 端功能；第二，完成覆盖用户、商品、订单、举报、运营内容和系统配置的管理后台；第三，建立清晰的数据库模型、REST API 与权限边界；第四，通过功能、接口、权限、兼容性和性能测试验证系统可用性；第五，总结当前原型在真实认证、并发锁定、支付和售后等方面的边界。

2.5 可提炼的特色与创新点

创新点应克制表达，建议写成“设计特色”而不是夸大为算法创新。

- 面向校园二手交易场景设计同校信息、校区取货地点、实名认证状态、信用分和面交安全提示。
- 将 C 端交易流程与管理后台审核、举报、公告、轮播、分类、配置和审计日志连接起来，形成双端协同。
- 使用订单状态机描述待处理、已付款、待面交、已完成、取消、退款和争议等业务状态，使交易过程可追踪。
- 以商品为上下文组织会话，减少聊天内容与交易对象脱节的问题。

- 使用 Vue 3 响应式前端与 FastAPI REST 服务实现前后端分离，并通过 SQLAlchemy 完成领域实体映射。
- 使用 ECharts 展示近七日交易趋势和商品分类分布，为后台运营提供数据支持。

2.6 论文必须声明的实现边界

- 当前付款操作是校园面交业务状态模拟，没有接入第三方资金支付与真实账务系统。
- 当前校园认证属于演示级状态，不应声称已完成权威学籍系统核验。
- 当前消息使用 REST 接口读写，不应声称已经通过 WebSocket 实现真正实时推送。
- SQLite 适合课程设计和单机演示，生产环境应迁移到 PostgreSQL 或 MySQL 并补充并发控制。
- 测试章节只能填写实际测得的数据，未测数据必须保留为待完成项。

三、全文规模与章节分配

3.1 建议篇幅

建议正文控制在 25,000—32,000 个汉字左右，图 25—35 幅，表 18—25 个。该范围仅用于规划，最终以学校要求为准。

内容	建议字数	主要任务
中文摘要与英文摘要	900—1,300	概括问题、方法、系统、测试和结论
第 1 章 绪论	3,000—4,000	建立研究问题、现状、目标和论文结构
第 2 章 相关技术	3,000—4,000	说明真实采用的前后端、数据与安全技术
第 3 章 需求分析	4,000—5,000	描述角色、流程、功能与非功能需求
第 4 章 系统设计	5,500—7,000	架构、模块、状态、数据库、接口与安全设计
第 5 章 系统实现	6,500—8,500	展示关键模块、处理流程、页面与关键代码
第 6 章 系统测试	4,000—5,500	功能、接口、权限、性能和兼容性验证
第 7 章 总结与展	1,500—2,000	总结贡献、局限和后续方向

内容	建议字数	主要任务
望		

3.2 建议叙事链路

绪论提出校园交易问题；相关技术解释解决问题所需工具；需求分析明确系统必须做什么；系统设计说明准备怎样做；系统实现证明已经做出来；系统测试证明做出的系统满足需求；总结与展望说明完成程度和仍然存在的边界。

四、前置部分框架

4.1 封面

严格复用学校模板，字段通常包括学校名称、论文类型、题目、学号、专业、学生姓名、指导教师和完成日期。论文题目在封面、摘要、诚信声明、目录页眉和评定表中必须完全一致。

4.2 诚信声明

直接使用学校固定文本，不自行修改法律表述。签名和日期应在最终提交版中处理。

4.3 中文摘要

建议 500—700 字，使用单段或 3—4 个自然段，按照五步结构组织。

第一步写背景与问题：校园闲置商品具有数量大、时效强、低客单价和校内面交等特点，微信群、QQ 群和线下公告存在信息分散、检索困难与交易留痕不足。

第二步写研究方法：采用软件工程方法开展需求分析、架构设计、数据库设计与模块实现，使用 UML 或业务流程图描述角色与流程。

第三步写技术方案：前端采用 Vue 3、Vue Router、Pinia、Axios、Element Plus 与 ECharts，后端采用 FastAPI、SQLAlchemy 和 SQLite，采用基于令牌的身份认证与 REST API。

第四步写实现成果：系统实现 C 端商品浏览、筛选、发布、收藏、聊天、订单、评价、通知和举报，并实现管理端用户、商品、订单、风控、运营内容、配置与审计日志。

第五步写测试结论：只能填写真实测试结果，例如功能用例通过率、关键接口平均响应时间、并发量、错误率和响应式页面验证结果。未完成性能测试前不要写“满足高并发”。

4.4 中文关键词

建议使用 4—5 个关键词：校园二手交易；前后端分离；FastAPI；Vue 3；信息管理系统。关键词避免使用“Python”和“系统”这类过宽的单词单独占位。

4.5 英文摘要与关键词

英文摘要内容必须与中文摘要一一对应，不额外增加中文摘要没有的性能数据。推荐题目译为：Design and Implementation of a Campus Second-hand Goods Trading Platform Based on FastAPI and Vue。关键词可写为 Campus second-hand trading; Front-end and back-end separation; FastAPI; Vue 3; Information management system。

4.6 目录、图目录与表目录

目录至少显示到三级标题。图、表超过 15 个时建议单独生成图目录和表目录。所有图表必须在正文中先引用后出现，格式统一为“如图 4-3 所示”“见表 6-2”。

五、可直接采用的论文目录

第 1 章 绪论

1.1 研究背景

1.2 国内外研究现状

1.2.1 国外校园二手交易平台研究与应用现状

1.2.2 国内校园二手交易平台研究与应用现状

1.2.3 现有研究不足与本文切入点

1.3 研究目的与意义

1.3.1 研究目的

1.3.2 理论意义

1.3.3 实践意义

1.4 研究内容与研究方法

1.4.1 主要研究内容

1.4.2 研究方法

1.4.3 技术路线

1.5 论文组织结构

1.6 本章小结

第 2 章 系统相关技术

2.1 B/S 架构与前后端分离模式

2.2 Vue 3 前端框架

2.3 Vue Router、Pinia 与 Axios

2.4 Element Plus 与 ECharts

2.5 FastAPI Web 框架

2.6 SQLAlchemy ORM 与 SQLite 数据库

2.7 RESTful API 设计

2.8 PBKDF2 密码哈希与令牌认证

2.9 本章小结

第 3 章 系统需求分析

3.1 系统建设目标

3.2 系统可行性分析

3.2.1 技术可行性

3.2.2 经济可行性

3.2.3 操作可行性

3.2.4 合规与安全可行性

3.3 系统角色分析

3.3.1 游客角色

3.3.2 校园用户角色

3.3.3 管理员角色

3.4 系统业务流程分析

3.4.1 用户注册与登录流程

3.4.2 商品发布与审核流程

3.4.3 商品浏览与沟通流程

3.4.4 下单与面交流程

3.4.5 举报与处理流程

3.5 C 端功能需求

3.5.1 账户与个人中心需求

3.5.2 首页与商品检索需求

3.5.3 商品发布与管理需求

3.5.4 收藏与消息需求

3.5.5 订单与评价需求

3.5.6 通知与举报需求

3.6 管理后台功能需求

3.6.1 数据概览需求

3.6.2 用户管理需求

3.6.3 商品审核需求

3.6.4 订单管理需求

3.6.5 举报与风控需求

3.6.6 运营内容管理需求

3.6.7 系统设置与审计需求

3.7 非功能性需求

3.7.1 性能需求

3.7.2 安全需求

3.7.3 易用性需求

3.7.4 兼容性与响应式需求

3.7.5 可维护性与可扩展性需求

3.8 本章小结

第 4 章 系统总体设计

4.1 设计目标与原则

4.2 系统总体架构设计

4.2.1 前端表现层

4.2.2 API 与业务处理层

4.2.3 数据持久层

4.2.4 静态资源与上传文件层

4.3 系统功能模块设计

4.3.1 C 端功能模块设计

4.3.2 管理后台功能模块设计

4.4 核心业务流程与状态设计

4.4.1 身份认证与权限流程

4.4.2 商品生命周期设计

4.4.3 订单状态机设计

4.4.4 会话与消息流程设计

4.4.5 举报处理状态设计

4.5 数据库设计

4.5.1 数据库概念结构设计

4.5.2 数据库逻辑结构设计

4.5.3 核心数据表设计

4.5.4 数据完整性与索引设计

4.6 REST API 设计

4.6.1 API 分组与统一约定

4.6.2 认证与用户接口

4.6.3 商品与收藏接口

4.6.4 订单与评价接口

4.6.5 消息、通知与举报接口

4.6.6 管理后台接口

4.7 权限与安全设计

4.8 前端界面与响应式设计

4.9 异常处理、配置与审计设计

4.10 本章小结

第 5 章 系统详细实现

5.1 开发与运行环境

5.2 项目结构与启动流程

5.3 用户注册登录与权限控制实现

5.4 首页推荐与商品市场实现

5.5 商品详情、收藏与举报实现

5.6 商品发布、编辑与审核实现

5.7 商品会话与消息中心实现

5.8 订单创建与状态流转实现

5.9 评价、信用与通知实现

5.10 管理后台数据概览实现

5.11 管理后台业务管理实现

5.11.1 用户状态管理实现

5.11.2 商品审核管理实现

5.11.3 订单查询管理实现

5.11.4 举报处理实现

5.11.5 分类、轮播与公告管理实现

5.11.6 系统配置与审计日志实现

5.12 文件上传与安全机制实现

5.13 响应式布局与交互体验实现

5.14 本章小结

第 6 章 系统测试

6.1 测试目标与测试策略

6.2 测试环境与工具

6.3 功能测试

6.3.1 注册登录功能测试

6.3.2 商品浏览与筛选功能测试

6.3.3 商品发布与审核功能测试

6.3.4 收藏与消息功能测试

6.3.5 订单与评价功能测试

6.3.6 管理后台功能测试

6.4 API 与权限测试

6.5 性能与并发测试

6.6 安全性测试

6.7 兼容性与响应式测试

6.8 测试结果与缺陷分析

6.9 本章小结

第 7 章 总结与展望

7.1 研究工作总结

7.2 系统不足

7.3 未来展望

参考文献

致谢

附录 A 主要 API 清单

附录 B 核心数据库表结构

附录 C 完整测试用例

附录 D 系统部署与运行说明

六、各章节详细写作说明

第 1 章 绪论

1.1 研究背景

写作目的：说明为什么需要建设校园二手商品交易平台，使研究问题从宏观环境逐步落到具体校园场景。

建议展开顺序：先写循环经济、绿色校园和大学生消费结构，再写校园闲置物品类型及交易特点，随后分析微信群、QQ 群、表白墙、线下跳蚤市场的局限，最后引出统一平台建设需求。

必须覆盖的问题：信息发布位置分散；商品信息缺少统一字段；搜索和筛选困难；交易对象身份与信用难核验；聊天、订单和面交过程缺少记录；违规商品和纠纷处理依赖人工；学校运营人员无法获得可分析数据。

建议证据：教育数字化、绿色低碳、循环经济与二手交易行业的权威政策或研究数据；大学生二手交易行为相关论文；校园二手市场调查数据。若没有自行问卷，不得虚构“多数学生认为”等比例结论。

建议图表：可选“图 1-1 传统校园闲置交易问题示意图”。本节不宜放系统界面截图。

建议字数：900—1,100 字。

1.2 国内外研究现状

1.2.1 国外研究现状

从三条线索综述：第一，C2C 二手电商平台和共享经济研究；第二，信任、声誉、内容治理与交易安全机制；第三，前后端分离 Web 平台和教育场景数字化服务。每类选取有代表性的英文论文、正式出版物或权威平台资料。

不要把国外商业产品介绍写成学术综述。每一段至少完成“研究对象—使用方法—主要结论—对本文启示”的闭环。

1.2.2 国内研究现状

可按照技术和业务两类组织。技术类包括 Spring Boot、Django、Flask、FastAPI、Vue、微信小程序等校园交易系统；业务类包括身份认证、推荐、信用机制、交易流程、内容审核和校园场景适配。

国内研究不应简单罗列“某某设计了某系统”。应比较系统角色、核心功能、技术架构、测试方式和不足，并形成小结。

1.2.3 现有研究不足与本文切入点

建议归纳为五点：重展示轻交易闭环；C 端与管理端协同不足；身份、信用和风控设计较弱；消息、订单、审核与举报之间缺乏统一状态；测试往往只做功能截图，缺少权限、接口和响应式验证。

本文切入点应写为：针对校园同校交易场景，构建包含 C 端和管理后台的双端系统，以前后端分离架构、订单状态管理、内容审核、举报处理和运营可视化形成完整平台。

建议表格：“表 1-1 相关研究对比”，字段可包含作者与年份、研究对象、技术路线、核心功能、验证方法、主要不足。所有条目必须真实可检索。

建议字数：1,200—1,600 字，引用 12—18 篇文献。

1.3 研究目的与意义

1.3.1 研究目的

用可验证的目标表述，不写空泛口号。目标包括建立统一商品信息入口、支持完整交易流程、提供校园特征字段、构建后台治理能力、形成可测试的前后端系统。

1.3.2 理论意义

理论意义应保持克制：为前后端分离技术在校园二手交易信息系统中的应用提供案例；为校园交易角色、状态和数据模型设计提供参考。不要声称填补重大理论空白。

1.3.3 实践意义

分别从学生、管理员和学校角度写：学生提升闲置流转效率；管理员提升审核、举报和运营管理能力；学校形成可追踪、可统计、相对安全的校园服务载体。

1.4 研究内容与研究方法

1.4.1 主要研究内容

按需求、设计、实现、测试四部分概述。内容要和后文章节一一对应。

1.4.2 研究方法

可采用文献研究法、需求分析法、面向对象建模法、原型迭代法、软件测试法。每个方法说明在本文中的具体用途，不要只写定义。

1.4.3 技术路线

技术路线建议绘制为：问题识别 → 文献与需求调研 → 角色和流程分析 → 技术选型 → 架构与数据库设计 → C 端开发 → 管理后台开发 → 接口联调 → 功能与非功能测试 → 总结改进。

建议图："图 1-2 论文研究技术路线图"。

1.5 论文组织结构

用一段话逐章说明任务，每章 1—2 句。不要重复目录标题后不做解释。

1.6 本章小结

用 150—200 字概括本章已完成的论证，并自然过渡到第 2 章技术基础。

第 2 章 系统相关技术

本章原则：每节按照"技术定义—核心机制—选型原因—在本系统中的用途"写作。禁止大段复制官网，不介绍未使用的 Spring、Redis、Nginx、Docker 或微服务，除非之后确实完成对应实现与测试。

2.1 B/S 架构与前后端分离模式

说明浏览器、Vue 前端、FastAPI 后端和数据库的调用关系；比较前后端分离与服务端模板渲染的差异；解释其在多页面复用、独立开发、API 统一和响应式适配方面的优势。

建议图："图 2-1 前后端分离 B/S 架构示意图"。

2.2 Vue 3 前端框架

介绍组件化、组合式 API、响应式状态和单文件组件。结合本项目说明页面视图、公共组件 ProductCard/PageHeader、CLayout/AdminLayout 以及路由页面如何复用。

2.3 Vue Router、Pinia 与 Axios

Vue Router 负责 C 端和管理端路由、登录与管理员守卫；Pinia 保存令牌和当前用户；Axios 统一配置 /api 基础路径、请求头和异常拦截。可以用一个简短流程说明用户访问受限页面时的处理过程。

2.4 Element Plus 与 ECharts

Element Plus 用于表单、弹窗、表格、分页、标签和消息提示；ECharts 用于近七日订单/交易额趋势以及商品分类分布。说明组件库对一致性和开发效率的作用，同时强调业务逻辑仍由项目代码完成。

2.5 FastAPI Web 框架

介绍基于 Python 类型注解的参数校验、依赖注入、路由声明、Pydantic 数据模型和自动 API 文档。结合 get_current_user、get_admin_user、请求模型与 49 个 API 接口说明其应用。

2.6 SQLAlchemy ORM 与 SQLite 数据库

说明实体映射、会话、关系、查询构造和事务概念。解释当前为何选择 SQLite 作为单机开发数据库，并说明生产环境下的并发局限及迁移方案。不要把 SQLite 描述为大规模高并发数据库。

2.7 RESTful API 设计

说明资源路径、HTTP 方法、状态码、JSON 响应和前后端解耦。选取商品、订单和举报接口举例，解释 GET、POST、PUT、PATCH、DELETE 的语义。

2.8 PBKDF2 密码哈希与令牌认证

说明密码不以明文保存；PBKDF2 使用随机盐和多次迭代；登录成功后签发带用户、角色、签发时间和过期时间的 HMAC 令牌；前端在请求头中携带 Bearer Token；后端依赖函数校验用户和角色。

本节必须说明生产环境必须更换默认密钥，并可在展望中提出刷新令牌、撤销列表、登录限流和多因素认证。

2.9 本章小结

回扣技术如何共同支撑系统，不再重复概念定义。

第 3 章 系统需求分析

3.1 系统建设目标

从统一性、完整性、可信性、易用性和可管理性五个角度定义目标。建议给出系统范围：面向单校园或多校区学生，支持单件二手商品信息交易，不包含真实资金清算和快递物流系统。

3.2 系统可行性分析

3.2.1 技术可行性

论证 Vue、FastAPI、SQLAlchemy 和 SQLite 的成熟度；说明前后端开发、REST 联调、响应式布局和图表展示均已有可运行实现作为依据。

3.2.2 经济可行性

说明核心框架开源、开发环境成本低、单机演示无需商业软件许可。不得虚构精确经济收益，可采用定性分析。

3.2.3 操作可行性

分析学生对浏览器和移动网页的熟悉程度；说明系统采用清晰导航、统一表单、状态标签和反馈提示降低学习成本。

3.2.4 合规与安全可行性

分析个人信息、学生身份、聊天内容、商品图片和交易记录的敏感性；提出最小化收集、访问控制、密码哈希、审计和内容治理要求；明确真实上线需要隐私政策和校园管理授权。

3.3 系统角色分析

3.3.1 游客角色

允许访问首页、市场、商品详情和登录注册；收藏、联系卖家、下单、发布等操作必须登录。

3.3.2 校园用户角色

同时具有买家和卖家身份。买家可搜索、收藏、沟通、创建订单、推进交易、评价和举报；卖家可发布、编辑、下架商品，接收会话和订单，并查看交易状态。

3.3.3 管理员角色

具有数据概览、用户状态管理、商品审核、订单查询、举报处理、运营内容管理、系统配置和审计日志权限。

建议图：“图 3-1 系统总体用例图”“图 3-2 校园用户用例图”“图 3-3 管理员用例图”。用例名称必须使用动宾结构。

3.4 系统业务流程分析

3.4.1 用户注册与登录流程

注册输入用户名、校园邮箱、密码、昵称、学校和学号；后端校验唯一性并哈希密码；登录支持用户名、邮箱或手机号；账号被暂停或封禁时拒绝登录；登录后根据角色进入 C 端或后台。

3.4.2 商品发布与审核流程

用户上传图片并填写商品字段；后端根据配置将普通用户商品置为待审核；管理员审核通过后发布，驳回时写入原因并通知卖家；商品修改后重新进入审核；卖家可主动下架。

3.4.3 商品浏览与沟通流程

用户按分类、关键词、价格、成色和排序检索；查看详情后可收藏或建立围绕该商品的会话；同一买家、卖家和商品组合复用一个会话。

3.4.4 下单与面交流程

买家确认见面地点并创建订单；订单依次经历待处理、已付款、待面交和已完成，也可能取消、退款或产生争议；完成订单后商品置为已售，并允许双方评价。

应特别说明当前为业务状态模拟，真实上线需增加卖家接单、商品唯一锁定和取货码核销。

3.4.5 举报与处理流程

用户针对商品、用户或订单提交原因与说明；管理员受理并标记处理中；完成核查后解决或驳回；处理结果以通知形式反馈举报人，并记录管理员审计日志。

建议图：“图 3-4 商品发布审核活动图”“图 3-5 订单交易活动图”“图 3-6 举报处理活动图”。

3.5 C 端功能需求

3.5.1 账户与个人中心需求

包括注册、登录、退出、当前用户查询、资料修改、校园身份状态、信用分、我的发布、收藏、订单和通知。

3.5.2 首页与商品检索需求

包括轮播内容、公告、安全提示、商品分类、推荐商品、全局搜索、结果内搜索、价格区间、成色、分类、多种排序和分页。

3.5.3 商品发布与管理需求

包括最多九张图片、封面、标题、分类、描述、售价、原价、成色、学校、取货地点和交易方式；支持创建、编辑、重新审核和下架。

3.5.4 收藏与消息需求

支持收藏切换、收藏列表、围绕商品发起会话、会话列表、未读数、历史消息和文本发送。

3.5.5 订单与评价需求

支持买入/卖出筛选、状态筛选、订单详情、付款、确认面交、完成、取消、退款、争议和 1—5 星评价。

3.5.6 通知与举报需求

支持订单、商品审核、举报进展等通知，以及商品、用户和订单举报。

建议表：“表 3-1 C 端功能需求表”，字段为编号、模块、功能点、输入、输出、前置条件、优先级。

3.6 管理后台功能需求

分别描述数据概览、用户管理、商品审核、订单查询、举报风控、分类/轮播/公告、系统配置和管理员操作日志。每节必须说明查询条件、管理动作、输出状态和通知对象。

建议表：“表 3-2 管理后台功能需求表”。

3.7 非功能性需求

3.7.1 性能需求

先定义待验证指标，例如普通查询接口 P95 响应时间、页面首屏时间、并发用户数和错误率。指标应结合 SQLite 单机环境合理设置，最终数值由第 6 章测试确定。

3.7.2 安全需求

包括密码哈希、令牌过期、角色鉴权、资源所有权校验、上传类型限制、管理员操作审计、个人信息脱敏和错误信息控制。

3.7.3 易用性需求

包括一致导航、清晰状态、表单提示、成功/失败反馈、空状态、加载状态和重要操作二次确认。

3.7.4 兼容性与响应式需求

系统应适配常见桌面浏览器和 390px 左右移动视口，不出现横向溢出，核心操作在移动端仍可访问。

3.7.5 可维护性与可扩展性需求

前端按布局、视图、组件、状态和 API 分层；后端按模型、序列化、安全、数据库和路由组织；配置项集中管理；API 具备一致路径和错误返回。

3.8 本章小结

总结系统范围、三类角色、五条核心业务流程和功能/非功能需求，并引出总体设计。

第 4 章 系统总体设计

4.1 设计目标与原则

原则包括前后端职责分离、领域对象清晰、最小权限、状态可追踪、页面组件复用、接口统一和渐进扩展。说明为何使用单体 REST 服务而非微服务：系统规模适中，单体结构更便于课程项目部署和维护。

4.2 系统总体架构设计

4.2.1 前端表现层

Vue 页面通过 Vue Router 划分 C 端与后台路由；Pinia 保存认证状态；Axios 访问后端；Element Plus 和自定义 CSS 完成交互与响应式布局。

4.2.2 API 与业务处理层

FastAPI 接收请求，Pydantic 校验参数，依赖函数完成身份和角色校验，路由函数执行业务规则，序列化函数生成 JSON 响应。

4.2.3 数据持久层

SQLAlchemy ORM 将 14 个实体映射到 SQLite，Session 管理查询和提交；关键字段建立唯一约束或索引。

4.2.4 静态资源与上传文件层

FastAPI 通过 /uploads 提供上传图片访问，前端公共素材位于 public/assets。说明真实生产环境应使用对象存储和 CDN。

建议图：“图 4-1 系统总体架构图”“图 4-2 前端组件与路由结构图”“图 4-3 后端请求处理流程图”。

4.3 系统功能模块设计

4.3.1 C 端功能模块设计

建议划分为账户中心、内容发现、商品管理、收藏管理、消息沟通、订单交易、评价信用、通知举报八个模块。

4.3.2 管理后台功能模块设计

建议划分为数据概览、用户管理、商品治理、订单管理、举报风控、运营管理、系统设置和操作审计八个模块。

建议图：“图 4-4 C 端功能模块图”“图 4-5 管理后台功能模块图”。

4.4 核心业务流程与状态设计

4.4.1 身份认证与权限流程

说明注册密码哈希、登录令牌签发、前端保存令牌、Axios 注入请求头、后端解析令牌、查询用户状态和管理员角色判断。分别画普通登录与管理员访问后台的时序图。

4.4.2 商品生命周期设计

商品状态包括 pending、published、rejected、off_shelf、sold。说明发布、审核、编辑、下架、恢复和订单完成对状态的影响。

4.4.3 订单状态机设计

订单状态包括 pending、paid、meeting、completed、cancelled、refunding、disputed。说明允许的状态迁移、操作者和副作用，例如完成后更新 completed_at 并将商品置为已售。

4.4.4 会话与消息流程设计

会话由商品、买家和卖家唯一确定；消息记录发送者、内容、类型、已读状态和时间；读取会话消息时更新对方未读消息。

4.4.5 举报处理状态设计

举报状态包括 pending、processing、resolved、rejected；处理时记录管理员、结果和完成时间，并产生通知和审计日志。

建议图：“图 4-6 登录认证时序图”“图 4-7 商品状态图”“图 4-8 订单状态机图”“图 4-9 消息发送时序图”“图 4-10 举报处理时序图”。

4.5 数据库设计

4.5.1 数据库概念结构设计

E-R 图至少展示用户、商品、分类、收藏、订单、会话、消息、评价、举报和通知之间的关系；运营内容和配置可单独显示，避免一张图过密。

4.5.2 数据库逻辑结构设计

当前系统主要实体如下。

实体	核心作用	主要关系
User	用户、管理员、校园信息、信用与状态	发布商品，参与订单、会话、评价和举报
Category	商品分类和前台展示属性	一对多关联 Product
Product	商品信息、价格、成色、地点、图片和状态	属于用户和分类，被收藏、下单和咨询
Favorite	用户商品收藏关系	User 与 Product 多对多中间实体
Order	买卖双方、金额、状态、取货码和时间	关联 Product、Buyer、Seller
Conversation	以商品为上下文的买卖会话	关联商品、买家、卖家和多条消息
Message	会话消息与已读状态	属于 Conversation 和发送者
Review	订单完成后的双方评价	关联订单、评价人和被评价人
Report	针对商品、用户或订单的举报	关联举报人和处理管理员
Notification	订单、审核、举报和系统通知	属于 User
Banner	首页轮播内容	由管理员维护
Announcement	公告、安全提醒和活动内容	由管理员维护
AuditLog	管理员关键操作记录	关联操作管理员
SystemConfig	审核开关、图片数、服务费等配置	以键值方式保存系统参数

4.5.3 核心数据表设计

正文重点列出 User、Product、Order、Conversation、Message、Report 六张表，其他表可放附录。每张表字段说明至少包含字段名、数据类型、约束、是否为空、默认值和业务含义。

4.5.4 数据完整性与索引设计

说明用户名和邮箱唯一；收藏采用用户+商品唯一约束；会话采用商品+买家+卖家唯一约束；评价采用订单+评价人唯一约束；订单号唯一；常用查询状态、外键和创建时间建立索引。指出当前仍需增加同一商品进行中订单的数据库级唯一占用约束。

建议图表：“图 4-11 系统 E-R 图”“表 4-1 用户表”“表 4-2 商品表”“表 4-3 订单表”“表 4-4 会话表”“表 4-5 消息表”“表 4-6 举报表”。

4.6 REST API 设计

4.6.1 API 分组与统一约定

现有后端包含 49 个路由。说明 /api 前缀、JSON 输入输出、Bearer Token、HTTP 状态码、分页参数和统一异常消息。

4.6.2—4.6.6 接口分组

按认证与用户、商品与收藏、订单与评价、消息与通知、举报、管理后台六组列出代表接口。正文不必列出全部 49 个接口，建议正文展示 15—20 个关键接口，完整清单放附录 A。

建议表：“表 4-7 核心 API 设计”，字段为接口名称、方法、路径、权限、关键参数、返回结果。

4.7 权限与安全设计

建立游客、普通用户、管理员权限矩阵；区分路由访问控制、业务资源所有权控制和管理员角色控制；说明 PBKDF2、令牌签名、账号状态、上传扩展名和审计日志。诚实指出尚缺登录限流、刷新令牌、文件内容检测和真实认证。

建议表：“表 4-8 系统角色权限矩阵”。

4.8 前端界面与响应式设计

说明校园绿与象牙白视觉风格、CLayout/AdminLayout 双布局、公共组件、页面路由、桌面端信息密度和移动端重排策略。可展示首页桌面与 390px 移动端对比。

4.9 异常处理、配置与审计设计

说明 Axios 统一处理错误提示；后端通过 HTTPException 返回 400、401、403、404、409 等状态；SystemConfig 保存系统设置；关键管理动作写入 AuditLog；用户状态变化和审核结果产生 Notification。

4.10 本章小结

概括架构、模块、状态、数据、接口和权限设计如何承接第 3 章需求，并引出具体实现。

第 5 章 系统详细实现

本章统一采用七步写法：业务场景 → 页面入口 → 前端状态与交互 → API 调用 → 后端业务规则 → 数据变化与异常处理 → 运行截图。关键代码每处截取 10—20 行，重点解释逻辑，不整页粘贴代码。

5.1 开发与运行环境

建议表格列出 macOS 或实际开发系统、Python 3.12、Node.js、FastAPI、SQLAlchemy、Vue 3、Vite、SQLite、IDE、浏览器及版本。版本必须从实际环境或锁文件读取。

5.2 项目结构与启动流程

说明根目录、backend/app、frontend/src、public/assets、backend/uploads 和 scripts/dev.sh；说明后端启动时自动建表和生成演示数据，Vite 将 /api 和 /uploads 代理至后端。

建议图：“图 5-1 项目目录结构”“图 5-2 系统本地启动流程”。

5.3 用户注册登录与权限控制实现

前端 AuthView.vue 收集注册或登录数据；Pinia auth.js 保存令牌和用户；Axios 拦截器添加认证头；后端 register、login、get_optional_user、get_current_user、get_admin_user 依次完成校验。

重点解释用户名/邮箱重复冲突、密码哈希、账号状态、令牌过期、C 端与管理员重定向。展示登录页面和管理员路由守卫结果。

5.4 首页推荐与商品市场实现

首页接口一次返回轮播、公告、分类和商品；商品列表接口支持关键词、分类、成色、校区、最低价、最高价、排序、分页和收藏状态；前端 MarketView.vue 维护筛选条件并同步 URL 查询参数。

重点解释推荐排序、价格过滤、分页和登录用户收藏状态的批量查询。展示首页、市场筛选和无结果状态。

5.5 商品详情、收藏与举报实现

商品详情增加浏览次数，返回卖家、分类、收藏状态和同类商品；收藏接口根据已有记录执行新增或删除并同步计数；举报接口保存目标类型、目标 ID、原因和说明。

展示详情页、购买/联系/收藏按钮和举报弹窗。说明用户不能访问无权限的下架商品。

5.6 商品发布、编辑与审核实现

前端支持多图片上传、表单字段、创建和编辑；后端检查分类、序列化图片数组并根据 audit_required 设置待审核或发布状态；普通用户修改后重新审核；管理员审核时更新状态、审核意见、通知和审计日志。

本节应展示发布页、待审核状态、后台审核弹窗和审核通知，形成完整证据链。

5.7 商品会话与消息中心实现

联系卖家时创建或复用商品会话；会话列表按最后消息时间排序并计算未读数；读取消息后将对方消息置为已读；发送消息时更新会话最后消息和时间。

说明当前采用 REST 请求，页面“消息已实时同步”的文案仅表示界面数据同步，不等同于 WebSocket 实时推送；未来可加入 SSE/WebSocket。

5.8 订单创建与状态流转实现

创建订单前校验商品已发布、不能购买自己的商品、同一买家不存在进行中订单；生成订单号和六位取货码；状态接口通过迁移表限制操作；付款和完成记录时间；完成订单后商品置为已售；状态变化通知另一方。

本节核心是状态机，不要只展示订单页面。需解释每个动作允许的起始状态和操作者权限。

必须诚实讨论当前局限：不同买家可能同时创建同一商品订单；取货码尚未用于强制核销；支付是状态模拟。可将这些问题放在 7.2 系统不足并提出数据库锁定与卖家接单方案。

5.9 评价、信用与通知实现

仅已完成订单双方可以评价，同一用户对同一订单只能评价一次；通知覆盖订单、商品审核和举报进展；信用页展示校园认证、手机号和交易行为信息。说明信用分目前为展示性字段，尚未形成完整动态评分模型。

5.10 管理后台数据概览实现

后端聚合用户数、当日新增、商品数、待审核、订单数、进行中、GMV 和待处理举报；按日期聚合近七日订单和金额；统计分类商品数量；前端用 ECharts 绘制折线/柱状组合图和环形图。

解释 GMV 统计口径，只计入已付款、待面交和已完成状态。展示后台首页。

5.11 管理后台业务管理实现

5.11.1 用户状态管理实现

按关键词、状态和角色查询；管理员可恢复、暂停或封禁普通用户；禁止停用当前管理员；记录操作日志。

5.11.2 商品审核管理实现

支持关键词、分类、状态筛选；通过、驳回、下架、恢复与推荐；操作结果通知卖家并记录日志。

5.11.3 订单查询管理实现

按订单号和状态查询全平台订单；抽屉展示买卖双方、地点、留言、时间和取消/争议原因。说明当前后台为查询与辅助判断，尚未实现冻结、退款裁决等强干预。

5.11.4 举报处理实现

按状态和目标类型筛选；管理员将举报设为处理中、已解决或驳回；记录处理人、结论和时间；通知举报人。

5.11.5 分类、轮播与公告管理实现

分类支持名称、图标、颜色、排序和启停；轮播支持标题、图片、链接、排序和启停；公告支持类型、置顶和发布状态。

5.11.6 系统配置与审计日志实现

说明审核开关、图片数量、服务费等键值配置；说明管理员重要操作的主体、动作、目标、详情、IP 和时间。指出部分设置项当前仍为前端展示，需要后端规则或定时任务真正接入。

5.12 文件上传与安全机制实现

后端检查扩展名、生成时间戳和随机文件名、保存至上传目录并返回 URL；密码采用 PBKDF2；令牌使用 HMAC；后端在资源修改、订单、会话和后台接口中进行所有权或角色判断。

不要声称文件上传已完全安全。说明还应增加大小限制、MIME 内容检测、图片解码、病毒扫描、EXIF 清理和对象存储。

5.13 响应式布局与交互体验实现

说明桌面端首页六列商品、市场侧边筛选、消息双栏和后台侧栏；移动端通过媒体查询调整导航、网格和表单，验证 390px 视口无横向溢出。展示桌面首页与移动首页对比图。

5.14 本章小结

总结从账户、商品、消息、订单到后台治理的实现闭环，并引出第 6 章测试验证。

第 6 章 系统测试

6.1 测试目标与测试策略

测试目标包括需求覆盖、业务状态正确、权限边界有效、接口返回可靠、响应式布局可用以及基本性能满足演示环境。采用黑盒功能测试、API 测试、权限负向测试、浏览器兼容测试和负载测试。

6.2 测试环境与工具

列出硬件 CPU、内存、操作系统、Python、Node、浏览器、数据库、前后端启动参数、测试数据量和网络环境。工具可使用浏览器开发者工具、FastAPI Swagger、curl/HTTP 客户端、pytest、Locust 或 JMeter。只能写实际使用的工具。

6.3 功能测试

每个测试表字段统一为：用例编号、模块、前置条件、操作步骤、测试数据、预期结果、实际结果、结论。建议至少设计以下用例。

- 注册成功、用户名重复、邮箱重复、密码长度不合法。
- 普通用户登录、管理员登录、错误密码、停用账号登录。
- 分类筛选、关键词搜索、价格范围、成色筛选、排序和分页。
- 查看商品详情、收藏/取消收藏、查看下架商品权限。
- 发布必填校验、图片上传、待审核、编辑重新审核、主动下架。
- 发起会话、重复联系复用会话、发送空消息、未读消息更新。
- 创建订单、购买自己商品、重复订单、付款、确认面交、完成、取消和非法状态迁移。
- 已完成订单评价、重复评价、未完成订单评价。
- 商品举报、举报受理、处理结果通知。
- 管理员审核商品、停用用户、查询订单、处理举报、修改运营内容和查看审计日志。

正文选 15—20 个代表用例，完整用例放附录 C。建议表：“表 6-1 C 端功能测试用例”“表 6-2 管理后台功能测试用例”。

6.4 API 与权限测试

重点覆盖：无令牌访问受限接口返回 401；普通用户访问管理员接口返回 403；用户不能修改他人商品；非会话成员不能查看消息；非订单双方不能查看或操作订单；管理员不能停用自己；重复收藏和重复评价保持数据一致。

建议表：“表 6-3 API 与权限测试结果”。可在附录提供部分请求与响应示例。

6.5 性能与并发测试

建议选择首页、商品列表、商品详情、登录、会话列表和后台概览作为场景，设置 10、30、50、100 并发梯度。记录平均响应时间、P50、P95、最大响应时间、吞吐量、错误率、CPU 和内存。

测试前固定数据规模，例如 1,000 用户、10,000 商品、20,000 消息和 5,000 订单；若只使用演示数据，应明确结果只代表小数据集。SQLite 写并发能力有限，写接口性能应与读接口分开测试。

建议图表：“表 6-4 性能测试环境”“表 6-5 关键接口性能结果”“图 6-1 并发数与 P95 响应时间”“图 6-2 并发数与吞吐量”。不得预填理想结果。

6.6 安全性测试

测试非法或过期令牌、越权访问、错误密码、账号停用、上传非法扩展名、超长输入、SQL 注入样式输入、HTML/脚本输入和直接修改资源 ID。说明测试只能验证当前防护，不等同于专业渗透测试。

建议表：“表 6-6 安全性测试用例”。

6.7 兼容性与响应式测试

至少测试 Chrome、Edge 或 Safari 中实际可用的两个浏览器；测试 1440×1024、1366×768、390×844 等视口；检查导航、商品网格、表单、消息、订单和后台表格。记录是否出现横向滚动、遮挡、图片变形和不可点击控件。

6.8 测试结果与缺陷分析

统计计划用例数、执行数、通过数、失败数、修复后通过数和遗留问题；按账户、商品、消息、订单、后台和响应式模块统计缺陷。每个遗留问题说明严重程度、影响和后续计划。

建议图：“图 6-3 各模块缺陷数量”“图 6-4 缺陷严重程度分布”。测试结果不要只写“全部通过”，应展示修复过程和边界。

6.9 本章小结

用测试数据说明系统在当前环境下满足哪些需求、仍有哪些限制，为结论章节提供依据。

第 7 章 总结与展望

7.1 研究工作总结

按照“问题—方法—设计—实现—测试—价值”总结，不再逐页复述。明确完成 C 端和后台、14 个主要实体、49 个 API、订单状态、内容审核、举报处理和运营图表等成果。性能结论必须引用第 6 章实测数据。

7.2 系统不足

建议诚实写出：校园认证为演示级；同一商品缺少跨买家的数据库级锁定和卖家接单；取货码尚未强制核销；没有真实支付与退款清算；消息不是 WebSocket 实时推送；SQLite 不适合生产高并发；信用分缺少动态模型；上传安全、登录限流、隐私合规和自动化测试仍需加强。

7.3 未来展望

按优先级写，而不是随意罗列：第一阶段补实名认证、商品锁定、卖家接单、取货码核销和售后工单；第二阶段迁移 PostgreSQL、对象存储、Redis、异步任务和 WebSocket；第三阶段增加求购广场、教材 ISBN/课程筛选、毕业季批量发布、地图交接点、议价和降价提醒；第四阶段在合规前提下探索推荐、风险识别和运营分析。

七、建议图目录

图号	图名	所属章节	生成依据
图 1-1	传统校园闲置交易问题示意图	1.1	背景问题归纳
图 1-2	论文研究技术路线图	1.4	研究流程
图 2-1	前后端分离 B/S 架构示意图	2.1	Vue—FastAPI—数据库关系
图 3-1	系统总体用例图	3.3	游客、用户、管理员角色
图 3-2	校园用户用例图	3.3	C 端功能
图 3-3	管理员用例图	3.3	后台功能
图 3-4	商品发布审核活动图	3.4	商品与审核状态
图 3-5	订单交易活动图	3.4	订单状态流程
图 3-6	举报处理活动图	3.4	举报状态流程
图 4-1	系统总体架构图	4.2	当前代码架构
图 4-2	前端组件与路由结构图	4.2	router、layouts、views、components
图 4-3	后端请求处理流程图	4.2	FastAPI 依赖与 SQLAlchemy
图 4-4	C 端功能模块图	4.3	C 端八模块
图 4-5	管理后台功能模块图	4.3	后台八模块
图 4-6	登录认证时序图	4.4	AuthView、Pinia、Axios、FastAPI
图 4-7	商品生命周期状态图	4.4	五种商品状态
图 4-8	订单状态机图	4.4	七种订单状态
图 4-9	消息发送时序图	4.4	Conversation 与 Message
图 4-10	举报处理时序图	4.4	Report、Notification、AuditLog

图号	图名	所属章节	生成依据
图 4-11	系统 E-R 图	4.5	14 个实体
图 5-1	项目目录结构	5.2	当前工作区
图 5-2	系统启动流程	5.2	dev.sh 与前后端服务
图 5-3	C 端首页	5.4	实际截图
图 5-4	商品市场筛选页面	5.4	实际截图
图 5-5	商品详情与购买确认	5.5	实际截图
图 5-6	商品发布页面	5.6	实际截图
图 5-7	后台商品审核页面	5.6	实际截图
图 5-8	消息中心页面	5.7	实际截图
图 5-9	我的订单页面	5.8	实际截图
图 5-10	订单状态处理逻辑	5.8	关键代码或流程图
图 5-11	信用与认证页面	5.9	实际截图
图 5-12	管理后台数据概览	5.10	实际截图
图 5-13	举报与风控页面	5.11	实际截图
图 5-14	桌面端与移动端页面对比	5.13	响应式截图
图 6-1	并发数与 P95 响应时间	6.5	实测数据
图 6-2	并发数与吞吐量	6.5	实测数据
图 6-3	各模块缺陷数量	6.8	测试缺陷记录

图号必须按章连续，不重复；截图使用相同浏览器比例，裁掉无关桌面区域，确保文字可读。

八、建议表目录

表号	表名	用途
表 1-1	国内外相关研究对比	形成综述结论，不做文献堆砌
表 2-1	系统主要技术及用途	概括技术选型
表 3-1	C 端功能需求	对应需求编号
表 3-2	管理后台功能需求	对应需求编号
表 3-3	非功能性需求指标	为测试建立验收基线
表 4-1—4-6	六张核心数据表	字段、类型、约束与说明
表 4-7	核心 API 设计	路径、方法、权限和功能
表 4-8	系统角色权限矩阵	游客、用户、管理员权限
表 5-1	开发与运行环境	实际版本
表 5-2	项目主要目录及职责	代码结构说明
表 5-3	订单状态迁移规则	状态、动作、操作者、副作用
表 6-1	C 端功能测试用例	功能验证
表 6-2	管理后台功能测试用例	后台验证
表 6-3	API 与权限测试结果	负向和越权测试
表 6-4	性能测试环境	硬件、软件、数据量
表 6-5	关键接口性能结果	实测响应和吞吐
表 6-6	安全性测试用例	认证、越权、输入、上传
表 6-7	浏览器与视口兼容结果	响应式验证
表 6-8	缺陷修复统计	测试闭环

九、当前项目与论文证据映射

论文内容	当前证据位置	写作价值
技术栈与运行方式	README.md、requirements.txt、package.json	第 2 章和 5.1
前端路由与权限	frontend/src/router/index.js	角色路由和守卫
用户认证状态	frontend/src/stores/auth.js、backend/app/security.py	登录与令牌实现
API 请求封装	frontend/src/api/index.js	Axios 拦截与错误处理
C 端/后台布局	CLayout.vue、AdminLayout.vue	双端界面架构
首页与市场	HomeView.vue、MarketView.vue	推荐、筛选和分页
商品详情与发布	ProductDetailView.vue、PublishView.vue	商品流程
消息与订单	MessagesView.vue、OrdersView.vue	会话和状态机
管理后台	frontend/src/views/admin	审核、运营、风控和配置
14 个数据实体	backend/app/models.py	E-R 图和表结构
49 个 API	backend/app/main.py	接口设计与实现
序列化	backend/app/serializers.py	输出结构和隐私边界
演示数据	backend/app/seed.py	测试环境
页面验证证据	implementation-*.png、audit/feature-review-2026-08-12	实现截图与体验分析
构建与启动	scripts/dev.sh、Vite 配置	部署与运行说明

写作时不要只说“调用数据库完成查询”。应定位到具体实体、API 和页面，并说明请求参数、业务判断、返回结构和界面状态。

十、参考文献规划

10.1 文献数量与构成

建议准备 25—35 篇可核验文献，其中近五年文献占主要比例，英文文献不少于学校规定数量。参考文献可分为校园二手交易与循环经济、C2C 信任和信用、内容审核与交易安全、前后端分离架构、Vue/FastAPI/ORM、软件测试与可用性六组。

10.2 检索关键词

中文关键词：校园二手交易平台、大学生闲置交易、C2C 信任机制、二手电商信用评价、校园信息系统、FastAPI Web 系统、Vue 前后端分离、二手交易推荐、平台内容治理、软件系统测试。

英文关键词：campus second-hand trading platform、student recommerce、C2C trust mechanism、online marketplace reputation、second-hand e-commerce、FastAPI web application、Vue front-end architecture、marketplace content moderation、software usability testing。

10.3 来源优先级

优先使用中国知网、万方、Web of Science、IEEE Xplore、ACM Digital Library、ScienceDirect 等可检索论文；技术机制可引用 FastAPI、Vue、SQLAlchemy、OWASP 等官方文档。商业博客和问答网站只能用于开发参考，不宜作为主要学术文献。

10.4 引用规则

国内外研究现状的每个研究结论必须有引用；技术章节引用官方机制或权威论文；测试指标和行业数据注明来源；正文出现的每条参考文献都应在文末列出，文末每条文献也应在正文出现。不得直接沿用参考论文中的文献而不核验真实性。

十一、写作与开发补充任务

11.1 论文定稿前建议补充的开发工作

- 为同一商品增加跨买家的进行中订单唯一锁定或卖家接单机制。
- 将取货码改为实际核销条件，取消订单后隐藏或失效。
- 增加校园邮箱或管理员认证流程，避免填写学号即认证。
- 增加订单售后工单、证据和管理员裁决流程。
- 为系统设置中的自动取消、评价期限和敏感词开关补充真实后端规则。
- 增加 API 自动化测试、权限测试和数据初始化脚本。
- 为上传文件补充大小、MIME、图片解码和 EXIF 清理。
- 将生产密钥改为环境变量强制配置，增加登录限流。

这些工作并非全部必须在论文前完成，但 P0 交易一致性和自动化测试会显著提升论文质量。

11.2 论文材料采集顺序

第一步冻结论文版本和数据库；第二步生成架构、用例、状态和 E-R 图；第三步统一截取 C 端与后台页面；第四步导出数据库字段和 API 清单；第五步编写并执行测试用例；第六步采集性能原始数据；第七步整理缺陷与修复记录；第八步展开正文并逐节插入引用。

11.3 建议进度

阶段	主要产出	验收标准
第 1 阶段	文献检索、题目、目录、研究问题	文献可核验，目录三级编号统一
第 2 阶段	需求、用例与业务流程	每个模块有需求编号和角色
第 3 阶段	架构、状态、E-R、数据表和 API	设计与代码一致
第 4 阶段	实现章节与截图	每节有流程、代码和界面证据
第 5 阶段	测试环境、用例、性能和缺陷	所有数字来自实际执行
第 6 阶段	摘要、结论、参考文献和格式	前后一致，引用完整

十二、最终自检清单

12.1 内容一致性

- 题目、摘要、关键词、目录、结论中的系统名称一致。
- 技术章节出现的技术都能在代码或部署环境中找到。
- 需求、设计、实现和测试之间能够按功能逐项追踪。
- 商品、订单、举报和用户状态名称在全文保持一致。
- C 端与管理后台的角色权限描述不互相矛盾。

12.2 证据真实性

- 性能数据保留测试脚本、时间、环境和原始结果。
- 页面截图来自最终版本，无遮挡、无演示工具边框和隐私信息。
- 文献在正式数据库中可检索，作者、题目、期刊、年份和页码准确。
- 未实现功能明确写为不足或展望，不使用“已实现”措辞。
- 测试结论包含失败、修复和遗留项，而不是笼统“全部正常”。

12.3 图表规范

- 图题置于图下，表题置于表上，编号按章连续。
- 所有图表在正文中先引用，图表内容与叙述一致。
- 流程图箭头方向统一，状态图包含开始、迁移和结束条件。
- E-R 图基数关系正确，表结构字段与 models.py 一致。
- 截图文字在打印尺寸下仍可辨认。

12.4 语言与格式

- 避免“非常强大”“极大提高”“完美解决”等无证据表述。
- 第一次出现英文缩写时给出中文和英文全称。
- 代码标识使用等宽格式或学校规定格式，不大段粘贴源代码。
- 章末小结只总结本章，不提前写下一章结果。
- 参考文献采用学校指定的 GB/T 7714 或其他统一格式。

十三、框架落地后的下一步

按照本框架继续写作时，建议先完成第 3 章需求分析和第 4 章系统设计，因为这两章可直接从当前代码、页面和数据模型提取，且能反向发现需要补充的功能。随后完成第 6 章测试，得到真实数据后再写摘要、结论和绪论中的成果表述。摘要和结论应最后写，避免与最终实现不一致。